Revenue Leak Detection Engine

Identifying Hidden Revenue Loss Patterns

Dataset: Brazilian Olist E-Commerce Platform
Records: 99,441 orders | 550,000+ rows across 9 tables | R$ 13.6M GMV
Tools: Python | SQLite | pandas | scikit-learn Currency: Brazilian Real (R$)

Baibhav Dhakal (University of Wolverhampton)

Abstract

This report presents an end-to-end revenue leak detection analysis conducted on the Brazilian Olist e-commerce dataset, comprising 99,441 real orders totalling R$ 13.6M in gross merchandise value (GMV). Rather than applying conventional approaches such as churn prediction or sales forecasting, this project frames the analytical problem as an internal revenue audit: where is the business losing money silently, and how much?

The analysis is structured across three layers. The first layer uses Python and SQL to identify and quantify five distinct revenue leak pools through exploratory data analysis and targeted SQL queries employing window functions and common table expressions (CTEs). The second layer where we build a machine learning model i.e a Random Forest classifier which assigns a continuous leakage probability and projected dollar value to every transaction. The third layer synthesises the findings into an executive brief with three operational recommendations.

Baibhav Dhakal (University of Wolverhampton)

1. Introduction

1.1 Background and Motivation

The majority of projects applied to e-commerce datasets follow one of two framings: predicting which customers will churn, or forecasting future sales. Both are useful but both look forward. This project takes a different approach borrowed from management consulting and financial auditing: looking at what has already happened and asking where revenue was lost.

Revenue leakage refers to money that a business expected to receive but wasn't received under conditions that make reversal likely. It manifests in multiple forms: cancelled orders, late deliveries that trigger refunds, pricing mismatches where shipping costs exceed product value, and structural inefficiencies such as maintaining a large seller base that generates negligible revenue while consuming platform resources.

The goal is to produce an output that a finance director, operations manager, or equity analyst could act on directly, not just a model that data scientists find technically interesting.

1.2 Dataset

The Brazilian Olist dataset is a publicly available collection of real anonymised e-commerce transactions from 2016 to 2018. It contains nine relational tables covering orders, order items, payments, reviews, customers, sellers, products, geolocation, and product category translations. The dataset was chosen because it is large enough (550,000+ rows) and complex enough (multi-table relationships, real-world messiness) to support serious analysis, yet is rarely used for revenue leak framing in public portfolios.

Table 1.1 — Dataset Overview

Table	Rows	Key Columns
orders	99,441	order_id, order_status, delivery dates
order_items	112,650	order_id, product_id, seller_id, price, freight_value
payments	103,886	order_id, payment_type, payment_value, installments
reviews	99,224	order_id, review_score (1–5)
customers	99,441	customer_id, city, state
sellers	3,095	seller_id, city, state
products	32,951	product_id, category_name, dimensions
category_translation	71	category_name, english_name

1.3 Project Structure

The project is divided into four analytical sections corresponding to the structure of this report. Section 2 covers data acquisition, environment setup, and data quality assessment. Section 3 presents the exploratory data analysis and SQL-based leak detection. Section 4 describes the machine learning model. Section 5 presents findings and recommendations.

2. Data — Acquisition, Setup & Quality Assessment

2.1 Environment and Library Setup

We use the Jupyter Notebook environment using Anaconda. All nine CSV files were loaded into Python using pandas and then persisted into a local SQLite database. SQLite was chosen over alternatives such as PostgreSQL because it requires no server configuration;the entire database resides in a single portable file, which is appropriate for a self-contained analytical project.

```
# Environment setup
import pandas as pd
import numpy as np
import sqlite3
import os

print("pandas:", pd.__version__)
print("numpy:", np.__version__)
print("sqlite3:", sqlite3.version)
print("All libraries imported successfully ✓")
```

Code 2.1 — Library imports with version verification

Then we load all nine CSV files into named DataFrames.

```
base_path = r"C:\Users\dhaka\Desktop\macro_project\mero_revenue"

orders              = pd.read_csv(f"{base_path}\\olist_orders_dataset.csv")
order_items         = pd.read_csv(f"{base_path}\\olist_order_items_dataset.csv")
payments            = pd.read_csv(f"{base_path}\\olist_order_payments_dataset.csv")
reviews             = pd.read_csv(f"{base_path}\\olist_order_reviews_dataset.csv")
customers           = pd.read_csv(f"{base_path}\\olist_customers_dataset.csv")
sellers             = pd.read_csv(f"{base_path}\\olist_sellers_dataset.csv")
products            = pd.read_csv(f"{base_path}\\olist_products_dataset.csv")
category_translation =
pd.read_csv(f"{base_path}\\product_category_name_translation.csv")
```

Code 2.2 — Loading all CSV files into named DataFrames

Table 2.1 — Row Counts After Load

Table	Rows Loaded	Notes
orders	99,441	1:1 match with customers as expected
order_items	112,650	More than orders — multiple items per order
payments	103,886	More than orders — split payment methods
reviews	99,224	Slightly fewer — some orders unreviewed
customers	99,441	Clean 1:1 with orders
sellers	3,095	Small lookup table
products	32,951	Product catalogue
category_translation	71	71 product categories

After the tables are loaded they are put to SQL so that we can use SQL queries.

```
db_path = r"C:\Users\dhaka\Desktop\macro_project\mero_revenue\revenue_leak.db"
conn = sqlite3.connect(db_path)

# Write all tables to SQLite
for name, df in [("orders", orders), ("order_items", order_items),
                 ("payments", payments), ("reviews", reviews),
                 ("customers", customers), ("sellers", sellers),
                 ("products", products), ("category_translation",
category_translation)]:
    df.to_sql(name, conn, if_exists="replace", index=False)

print("Database created successfully ✓")
```

Code 2.3 — Persisting all DataFrames to SQLite

2.2 Schema and Data Type Assessment

After loading, the column names and data types of we inspect the tables. See the type of data they are in etc.. The most significant finding was that all date columns (purchase timestamp, delivery dates, estimated delivery date) were stored as string objects rather than datetime types. This required explicit conversion before any time-based calculations could be performed. All financial columns (price, freight_value, payment_value) were correctly typed as float64, confirming we could do revenue calculations without transformation.

2.3 Null Value Assessment

A null check was run across all eight tables. The results are summarised in Table 2.2. Critically, the order_items and payments tables which carry all revenue data contained zero nulls, confirming the integrity of the financial figures. The 2,965 null values in order_delivered_customer_date represent orders that were never delivered, which is not a data quality issue but a meaningful signal used in the leak analysis.

Table 2.2 — Null Value Summary (non-zero counts only)

Table	Column	Null Count	% of Table	Interpretation
orders	order_approved_at	160	0.2%	Minor — negligible impact
orders	order_delivered_carrier_date	1,783	1.8%	Orders that never left warehouse
orders	order_delivered_customer_date	2,965	3.0%	Undelivered orders — leak signal
reviews	review_comment_title	87,656	88.3%	Normal — most users only rate
reviews	review_comment_message	58,247	58.7%	Normal — optional field
products	product_category_name	610	1.9%	Labelled as 'uncategorized'

3. Methods — Exploratory Analysis & SQL Leak Detection

The analytical methodology proceeds in two stages. The first stage is exploratory data analysis (EDA) , a systematic investigation of the dataset to understand revenue distribution, identify anomalies, and form hypotheses about where leakage occurs.

The second stage translates those hypotheses into precise SQL queries using advanced techniques (CTEs and window functions) to produce exact, reportable figures.

3.1 Revenue Baseline

Before investigating leakage, we established the total revenue flowing through the platform from two independent sources: order_items (what was charged) and payments (what was collected). Comparing these two figures reveals any systematic discrepancy between invoiced and collected revenue.

```
total_gmv      = order_items['price'].sum()
total_freight  = order_items['freight_value'].sum()
total_revenue  = total_gmv + total_freight
total_payments = payments['payment_value'].sum()
discrepancy    = total_payments - total_revenue
avg_order_val  = order_items.groupby('order_id')['price'].sum().mean()
```

Code 3.1 — Revenue baseline calculation

Table 3.1 — Revenue Snapshot

Metric	Value
Total Product Revenue (GMV)	R$ 13,591,643.70
Total Freight Revenue	R$ 2,251,909.54
Total Revenue (GMV + Freight)	R$ 15,843,553.24
Total Payments Collected	R$ 16,008,872.12
Discrepancy	R$ 165,318.88
Average Order Value	R$ 137.75
Total Orders	99,441
Total Order Lines	112,650

The R$ 165,318 discrepancy between collected payments and GMV+freight is explained by installment interest and voucher values; it's not really a leak signal in isolation. The R$ 137.75 average order value serves as a baseline benchmark: sellers or categories significantly below this figure become candidates for structural leak investigation.

3.2 Order Status and Cancellation Analysis

Order status is the primary column for leak detection. Every non-delivered status represents revenue that either never arrived or is at risk of reversal. We first segmented all orders by status, then isolated cancelled and unavailable orders to quantify the confirmed leak pool.

```
# Order status breakdown
status_counts = orders['order_status'].value_counts()

# Revenue tied to non-delivered orders
non_delivered_ids  =
orders[~orders['order_status'].isin(['delivered'])]['order_id'].tolist()
revenue_at_risk    =
order_items[order_items['order_id'].isin(non_delivered_ids)]['price'].sum()

# Cancellation deep dive
cancelled_ids   = orders[orders['order_status'] == 'canceled']['order_id'].tolist()
unavailable_ids = orders[orders['order_status'] ==
'unavailable']['order_id'].tolist()
```

Code 3.2 — Order status segmentation and cancellation isolation

Table 3.2 — Order Status Breakdown

Status	Count	% of Orders	Revenue Tied
delivered	96,478	97.02%	—
shipped	1,107	1.11%	In transit
canceled	625	0.63%	R$ 95,235.27
unavailable	609	0.61%	R$ 2,007.69
invoiced	314	0.32%	Stuck in pipeline
processing	301	0.30%	Stuck in pipeline
created	5	0.01%	Abandoned
approved	2	0.00%	Abandoned

Table 3.3 — Cancellation Loss by Payment Type (SQL CTE Result)

Payment Type	Cancelled Orders	Total Lost (R$)	Avg Lost / Order	% of Total Loss
credit_card	444	97,375.31	219.31	67.97%
voucher	94	25,664.92	223.17	17.92%
boleto	95	17,504.10	184.25	12.22%
debit_card	7	2,711.27	387.32	1.89%

Here we can see our first leak pool. R$ 97,242.96 in cancellation and unavailability loss. Credit card cancellations account for 67.97% of this figure and carry additional chargeback processing costs beyond the product loss.

3.3 Customer Satisfaction Signal

Review scores (rated 1 to 5) are a leading indicator of refund requests and repeat cancellations. A customer who rates an order 1 or 2 stars has already expressed dissatisfaction and dissatisfied customers request refunds, file payment disputes, and do not return. We treated all 1–2 star orders as at-risk revenue.

```
# Low score isolation
low_score_ids      = reviews[reviews['review_score'] <= 2]['order_id'].tolist()
low_score_revenue  =
order_items[order_items['order_id'].isin(low_score_ids)]['price'].sum()
avg_low_score_val  =
order_items[order_items['order_id'].isin(low_score_ids)]['price'].mean()
avg_high_score_val =
order_items[~order_items['order_id'].isin(low_score_ids)]['price'].mean()
```

Code 3.3 — Isolating low-satisfaction orders and revenue

Table 3.4 — Review Score Distribution

Score	Count	% of Reviews	Interpretation
5 stars	57,328	57.78%	Satisfied — low leak risk
4 stars	19,142	19.29%	Satisfied — low leak risk
3 stars	8,179	8.24%	Neutral — monitor
2 stars	3,151	3.18%	Dissatisfied — at-risk revenue
1 star	11,424	11.51%	Highly dissatisfied — high leak risk

Here we can see our second leak pool. R$ 2,256,201.05 tied to 14,575 low-satisfaction orders. Notably, low-score orders had a higher average item value (R$ 125.03) than high-score orders (R$ 119.82), meaning leakage risk is concentrated in larger transactions.

3.4 Delivery Delay Analysis

Late delivery is one of the most measurable drivers of refund requests and chargebacks. We converted all date columns from string to datetime, then calculated the delay for every delivered order as the difference between actual and promised delivery date. Positive values indicate late delivery.

```
# Convert dates and calculate delay
orders['order_delivered_customer_date'] =
pd.to_datetime(orders['order_delivered_customer_date'])
orders['order_estimated_delivery_date'] =
pd.to_datetime(orders['order_estimated_delivery_date'])
orders['order_purchase_timestamp']      =
pd.to_datetime(orders['order_purchase_timestamp'])

delivered_orders = orders[orders['order_status'] == 'delivered'].copy()
delivered_orders['delay_days'] = (
    delivered_orders['order_delivered_customer_date'] -
    delivered_orders['order_estimated_delivery_date']
).dt.days
```

Code 3.4 — Date conversion and delay calculation

Table 3.5 — Delivery Performance Summary

Metric	Value
Total delivered orders	96,478
On-time or early	89,936 (93.2%)
Late deliveries	6,534 (6.8%)
Average delay (late orders only)	10.6 days
Median delay (late orders only)	7.0 days
Maximum delay	188 days
Revenue on late orders	R$ 985,924.34
Late revenue as % of GMV	7.3%

3.5 Delay-to-Dissatisfaction Correlation (SQL Window Function)

The most analytically significant query in the project. We bucketed every delivered order by delay magnitude and measured the average review score and low-satisfaction rate per bucket. This reveals the exact threshold at which late delivery destroys customer satisfaction — and therefore at which revenue becomes at risk.

```
WITH delay_buckets AS (
    SELECT order_id, review_score, delay_days,
        CASE
            WHEN delay_days <= 0              THEN '1_on_time_or_early'
            WHEN delay_days BETWEEN 1 AND 7   THEN '2_late_1_to_7_days'
            WHEN delay_days BETWEEN 8 AND 14  THEN '3_late_8_to_14_days'
            WHEN delay_days BETWEEN 15 AND 30 THEN '4_late_15_to_30_days'
            ELSE                                   '5_late_30_plus_days'
        END AS delay_bucket
    FROM order_delays
)
SELECT delay_bucket,
       COUNT(*)                    AS total_orders,
       ROUND(AVG(review_score),3)  AS avg_review_score,
       ROUND(100.0 * SUM(CASE WHEN review_score <= 2 THEN 1 ELSE 0 END)
             / COUNT(*), 2)        AS pct_low_score,
       ROUND(100.0 * COUNT(*) / SUM(COUNT(*)) OVER (), 2) AS pct_of_orders
FROM delay_buckets GROUP BY delay_bucket ORDER BY delay_bucket;
```

Code 3.5 — Delay bucket analysis using SQL window function SUM() OVER ()

Table 3.6 — Delay Bucket vs Customer Satisfaction

Delay Bucket	Orders	Avg Score	Low-Score Rate	% of All Orders
On-time or early	89,944	4.290	9.28%	93.35%
Late 1–7 days	3,612	2.713	49.42%	3.75%
Late 8–14 days	1,452	1.676	80.03%	1.51%
Late 15–30 days	1,014	1.617	81.66%	1.05%
Late 30+ days	331	2.054	67.98%	0.34%

We can see something interesting here, this is a critical threshold i.e just being 1–7 days late causes dissatisfaction to jump from 9.28% to 49.42% — a 5x increase. Beyond 8 days, 80% of customers give 1–2 stars. This single finding defines the intervention point for operational recommendations.

3.6 Freight Economics Analysis

For each order line, the freight-to-price ratio was calculated as freight_value divided by price. When this ratio equals or exceeds 1.0, the cost of shipping equals or exceeds the value of the product which is a structurally unprofitable transaction. Even at a ratio of 0.5, the unit economics are severely compressed.

```
order_items['freight_ratio'] = order_items['freight_value'] / order_items['price']

high_freight   = order_items[order_items['freight_ratio'] >= 1.0]
medium_freight = order_items[(order_items['freight_ratio'] >= 0.5) &
                             (order_items['freight_ratio'] < 1.0)]
```

Code 3.6 — Freight ratio calculation and threshold segmentation

Table 3.7 — Freight Economics Breakdown

Segment	Order Lines	Revenue	Avg Product Price	Avg Freight
Freight >= 100% of price (critical)	4,124	R$ 72,678.81	R$ 17.62	R$ 24.33
Freight 50–99% of price (warning)	14,820	R$ 496,404.41	—	—
All orders (baseline)	112,650	R$ 13.6M	R$ 120.65	R$ 19.98

Here we can identify another leak pool. R$ 569,083.22 on transactions with broken freight economics. The worst categories: electronics (611 lines), christmas supplies (freight ratio 0.367), food and drink categories (perishables with high shipping requirements).

3.7 Seller Performance Distribution (SQL NTILE Window Function)

The NTILE(10) window function split all 3,095 sellers into 10 equal revenue deciles. This is a standard consulting diagnostic for identifying platform resource drag: the proportion of sellers who consume infrastructure, support, and logistics costs while generating negligible revenue.

Table 3.8 — Seller Revenue Decile Analysis

Decile	Sellers	Total Revenue (R$)	Avg Item Price (R$)	Freight Ratio	% of GMV
1 (top 10%)	310	9,183,174	301.53	0.158	67.56%
2	310	2,060,449	336.21	0.181	15.16%
3	310	1,033,680	235.87	0.201	7.61%
4	310	550,185	207.76	0.200	4.05%
5	310	328,923	193.68	0.219	2.42%
6	309	202,310	154.58	0.226	1.49%
7	309	116,479	118.00	0.275	0.86%
8	309	66,790	106.20	0.274	0.49%
9	309	36,032	73.28	0.329	0.27%
10 (bot 10%)	309	13,617	34.87	0.592	0.10%

This confirms another leakpool. Bottom 50% of sellers (1,547 sellers) generate just 3.2% of GMV at an average of R$ 282 total revenue each. Freight ratio climbs from 0.158 in decile 1 to 0.592 in decile 10 — bottom-tier sellers have structurally unviable unit economics.

3.8 Master Risk Tier — Combined SQL Scoring

The final SQL query synthesises all signals into a single risk tier per order. Every delivered order is classified as critical, high, medium, or low based on the combination of delay days and review score. This framework becomes both the headline finding and the labelled dataset for the machine learning model.

```
WITH risk_scored AS (
    SELECT *,
        CASE
            WHEN delay_days > 14 AND review_score <= 2        THEN 'critical'
            WHEN delay_days BETWEEN 1 AND 14
                 AND review_score <= 2                        THEN 'high'
            WHEN delay_days > 0  AND review_score <= 3        THEN 'medium'
            WHEN review_score <= 2                            THEN 'medium'
            ELSE                                                   'low'
        END AS risk_tier
    FROM order_features
)
SELECT risk_tier,
```

```
        COUNT(DISTINCT order_id)              AS affected_orders,
        ROUND(SUM(price), 2)                  AS revenue_at_risk,
        ROUND(AVG(delay_days), 1)             AS avg_delay_days,
        ROUND(AVG(review_score), 2)           AS avg_review_score,
        ROUND(100.0 * SUM(price) /
              SUM(SUM(price)) OVER (), 2)     AS pct_of_gmv
FROM risk_scored GROUP BY risk_tier;
```

Code 3.7 — Master risk tier scoring with CASE logic and SUM() OVER() window function

Table 3.9 — Master Risk Tier Results

Risk Tier	Orders	Revenue at Risk (R$)	Avg Delay (days)	Avg Review Score	% of GMV
critical	1,047	167,681.76	26.2	1.11	1.27%
high	2,939	453,665.35	6.8	1.14	3.45%
medium	9,021	1,447,205.91	-11.6	1.37	10.99%
low	82,905	11,098,529.91	-12.2	4.59	84.29%

4. Machine Learning — Transaction-Level Risk Scoring

The SQL layer identified leak pools using rule-based logic we defined. The machine learning layer goes further: it learns the pattern from the data and assigns a continuous probability score to every transaction. The output is a dollar figure where each order receives a projected value at risk based on its characteristics.

4.1 Feature Engineering

A flat model table was constructed at the order level i.eone row per order, one column per signal. Features were selected based on their demonstrated relevance in the EDA and SQL analysis. Date columns were converted before feature extraction. The payment type column was integer-encoded (credit=1, debit=2, voucher=3, boleto=4) to make it usable by the model without one-hot encoding overhead.

Table 4.1 — Model Features

Feature	Type	Why Included
delay_days	Numeric	Primary leak driver — threshold identified in Section 3.5
fulfillment_days	Numeric	Total time from purchase to delivery
payment_installments	Numeric	Split payments correlate with cancellation complexity
payment_value	Numeric	Order size — larger orders have higher absolute risk
payment_type_encoded	Integer	Credit card cancellations are most costly
item_count	Numeric	Order complexity signal
total_price	Numeric	Revenue at stake per order
total_freight	Numeric	Absolute freight cost
avg_item_price	Numeric	Unit economics signal
freight_ratio	Numeric	Structural leak signal from Section 3.6
unique_sellers	Numeric	Multi-seller orders have higher fulfillment risk

```
# Risk label assignment (0=low, 1=medium, 2=high, 3=critical)
CASE
    WHEN delay_days > 14 AND review_score <= 2       THEN 3  -- critical
    WHEN delay_days BETWEEN 1 AND 14
         AND review_score <= 2                       THEN 2  -- high
    WHEN delay_days > 0  AND review_score <= 3       THEN 1  -- medium
    WHEN review_score <= 2                           THEN 1  -- medium
    ELSE                                                  0  -- low
END AS risk_label
```

Code 4.1 — Risk label assignment logic (mirrors SQL risk tier from Section 3.8)

Table 4.2 — Risk Label Distribution in Model Dataset

Label	Risk Tier	Count	% of Dataset
0	low	86,981	86.4%
1	medium	9,498	9.4%
2	high	3,059	3.0%
3	critical	1,103	1.1%

4.2 Model Selection and Training

A Random Forest classifier was selected for three reasons. First, it handles the significant class imbalance (86% low, 1% critical) through built-in class weighting without requiring synthetic oversampling. Second, it requires no feature scaling, unlike distance-based algorithms such as

logistic regression or support vector machines. Third, it produces feature importance scores that can be communicated to a non-technical business audience.

Class weights were computed using sklearn's compute_class_weight function with the 'balanced' setting, which automatically calculates weights inversely proportional to class frequency. This ensures the model pays proportionally more attention to the rare critical-tier orders. An 80/20 train/test split with stratification preserved the class distribution in both sets.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.utils.class_weight import compute_class_weight

# Deduplicate to one row per order
model_df = model_df.drop_duplicates(subset='order_id', keep='first')
# Result: 95,823 rows

X = model_df[feature_cols]
y = model_df['risk_label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Balanced class weighting
weights          = compute_class_weight('balanced', classes=np.array([0,1,2,3]),
y=y_train)
class_weight_dict = dict(zip([0,1,2,3], weights))

rf = RandomForestClassifier(
    n_estimators=200, max_depth=12, min_samples_leaf=10,
    class_weight=class_weight_dict, random_state=42, n_jobs=-1)

rf.fit(X_train, y_train)
```

Code 4.2 — Random Forest training with balanced class weighting

4.3 Model Performance

The model was evaluated on the held-out 20% test set (19,165 orders). The primary metric for this use case is recall on critical and high tiers because missing a genuine leak event by classifying it as safe is more costly than a false alarm. Both critical and high tiers achieved 100% recall, meaning the model never classified a serious leak as low-risk.

Table 4.3 — Classification Report

Risk Tier	Precision	Recall	F1-Score	Support
low	0.93	0.86	0.89	16,573
medium	0.22	0.31	0.26	1,796
high	0.58	1.00	0.73	587
critical	0.76	1.00	0.86	209
overall accuracy	—	0.81	—	19,165

Table 4.4 — Confusion Matrix

Actual \ Predicted	pred: low	pred: medium	pred: high	pred: critical
actual: low	14,241	1,989	306	37
actual: medium	1,092	556	119	29
actual: high	1	0	586	0
actual: critical	0	0	0	209

We can see that zero critical orders were predicted as low-risk. Zero high-risk orders were predicted as low-risk. The model's most important property is that it never misses a serious leak and it is confirmed by the confusion matrix.

4.4 Dollar Value Scoring

The model's probability outputs were converted into dollar figures using a weighted leak probability formula. The weights (0.25 for medium, 0.65 for high, 1.00 for critical) reflect the expected actual realisation rate per tier based on the EDA findings — medium-tier orders are early-warning signals and do not fully leak, while critical orders represent near-certain losses.

```
# Weighted leak probability
scored_df['leak_probability'] = (
    scored_df['prob_medium']   * 0.25 +
    scored_df['prob_high']     * 0.65 +
    scored_df['prob_critical'] * 1.00
)

# Dollar value at risk
scored_df['dollar_at_risk'] = scored_df['total_price'] *
scored_df['leak_probability']
```

Code 4.3 — Converting probability scores to dollar-at-risk figures

Table 4.5 — Transaction Risk Scoring Output

Risk Tier	Orders	Total Revenue (R$)	Dollar at Risk (R$)	Avg Leak Prob
critical	1,334	209,540.81	202,634.89	0.97
high	5,007	741,009.72	432,123.42	0.58
medium	12,601	2,548,060.27	442,490.12	0.17
low	76,881	9,609,998.68	970,579.44	0.10
TOTAL	95,823	13,108,609.48	2,047,827.88	0.156

5. Findings

This section consolidates all identified leak pools into a single summary, presents the two headline numbers, and establishes the basis for the recommendations in Section 6.

Table 5.1 — Complete Leak Pool Summary

Pool	Description	R$ Exposure	Confidence	Source
#1	Cancellations + unavailable orders	97,242.96	Confirmed	EDA + SQL
#2	Low satisfaction orders (1–2 star)	2,256,201.05	At-risk	EDA
#3	Late delivered orders (any delay)	985,924.34	At-risk	EDA
#4	Freight >= 50% of product price	569,083.22	At-risk	EDA + SQL
#5	Bottom 50% seller base drag	437,318.00	Structural	EDA + SQL
TOTAL EXPO SURE	All pools (overlap exists)	5,428,398.25	Upper bound	—
ML ESTIM ATE	Probability-weighted	2,047,827.88	Conservative	ML Model

Two numbers are reported intentionally. The total exposure figure of R$ 5,428,398 represents the upper bound — the sum of all identified at-risk revenue before accounting for overlap between pools. The model-estimated figure of R$ 2,047,827 is the conservative,

probability-weighted estimate that accounts for the fact that not every at-risk order will ultimately leak. The latter figure is the one used in executive communications.

Our model estimates the leakage to be R$ 2,047,827.88 i.e15.62% of GMV. Total upper-bound exposure: R$ 5,428,398.25 which is 39.9% of GMV. Both figures are derived from real transactions on a real platform using transaction-level analysis.

6. Recommendations

Three operational recommendations follow directly from the findings. Each is tied to a specific leak pool, an identifiable root cause, and a recoverable dollar amount. The recovery estimates use a conservative 30% realisation rate i.e we assume only 30 cents of every identified leak dollar is actually recovered through intervention.

Recommendation 1: Deploy Real-Time Delivery Delay Alerting

Root cause: Leak Pools #2 and #3. The delay-to-dissatisfaction analysis identified a precise intervention point: orders that cross from zero to one day late experience a 5x jump in low-satisfaction rate (9% to 49%). Every day of delay beyond that pushes the rate toward 80%.

Recommendation: Build an automated alert that flags orders the moment they cross the 1-day-late threshold. Proactive customer outreach at day 1 of delay i.e either an apology message, a delivery update or a small discount on the next order which will costs a fraction of processing a chargeback or refund dispute at day 14. The model already scores every order with a leak probability; critical and high-tier orders can be automatically escalated to an operations queue.

Table 6.1 — Delay Alert: Addressable Impact

Metric	Value
Revenue on late delivered orders	R$ 985,924
Conservative recovery rate	30%
Estimated annual recovery	R$ 295,777
Critical + high tier orders to monitor	3,986 orders

Recommendation 2: Implement Category-Level Freight Minimums

Root cause: Leak Pool #4. The SQL analysis identified four categories with structurally broken freight economics: electronics (611 lines where freight exceeds price), christmas supplies (freight ratio 0.367), food and drink, and signaling and security. In all cases, low-value items are being listed without accounting for the cost of shipping them.

Recommendation: Introduce minimum product price thresholds by category, or require sellers to absorb freight costs above a defined ratio cap (e.g. freight cannot exceed 40% of product price). This eliminates the 4,124 order lines where shipping costs more than the product and reduces the 14,820 lines in the warning zone.

Table 6.2 — Freight Policy: Addressable Impact

Metric	Value
Revenue on broken-economics order lines	R$ 569,083
Conservative recovery rate	30%
Estimated annual recovery	R$ 170,724
Order lines eliminated (freight > price)	4,124

Recommendation 3: Restructure the Bottom Seller Tier

Root cause: Leak Pool #5. The NTILE analysis showed that 1,547 sellers in the bottom 50% generate just R$ 437,318 in total revenue which is an average of R$ 282 each while consuming the same onboarding, support, payment processing, and logistics infrastructure as the top 309 sellers who generate 67.5% of GMV.

Recommendation: Introduce a minimum activity threshold for example, 10 completed orders or R$ 2,000 in revenue per rolling quarter, below which seller accounts are moved to an inactive status. This concentrates platform resources on high-performing sellers and eliminates the overhead of maintaining thousands of dormant low-value accounts.

Table 6.3 — Seller Restructure: Addressable Impact

Metric	Value
Revenue from bottom 50% sellers	R$ 437,318
Conservative recovery rate	30%
Estimated annual recovery	R$ 131,195
Sellers below R$ 1,000 total revenue	~1,100 sellers

Combined Recovery Estimate

Table 6.4 — Three-Intervention Recovery Summary

Recommendation	Addressable Leak (R$)	Conservative Recovery (30%)	Primary Leak Pool
1. Delay alerting system	985,924	295,777	Pools #2 + #3
2. Freight minimum policy	569,083	170,724	Pool #4
3. Seller tier restructure	437,318	131,195	Pool #5
TOTAL	1,992,325	597,696	—

At a conservative 30% recovery rate, three operational changes recover approximately R$ 597,696 annually without acquiring a single new customer, changing the product, or increasing marketing spend. The full model-estimated leakage of R$ 2,047,827 represents the ceiling if all interventions are executed at high effectiveness.